AFRILANG Stdlib

std/c/cryptcrc32

Bibliothèque AFRILANG `std/c/cryptcrc32` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : crc32, crc32Li

`import "std/c/cryptcrc32"` · `use cryptcrc32`

- `crc32(s text) ? number`
- `crc32List(data list number) ? list number`
- `verify(s text, expected number) ? bool`
- `update(crc number, byte number) ? number`
- `finalize(crc number) ? number`
- `chunkCrc(s text, chunkSize number) ? list number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/cryptcrc32` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : crc32, crc32Li

`import "std/c/cryptcrc32"` · `use cryptcrc32`

- `crc32(s text) ? number`
- `crc32List(data list number) ? list number`
- `verify(s text, expected number) ? bool`
- `update(crc number, byte number) ? number`
- `finalize(crc number) ? number`
- `chunkCrc(s text, chunkSize number) ? list number`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptcrc32"
use cryptcrc32
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

? crc32(s text) ? number
? crc32List(data list number) ? list
? verify(s text, expected number) ? bool
? update(crc number, byte number) ? number
? finalize(crc number) ? number
? chunkCrc(s text, chunkSize number) ? list

4. Exemples d'utilisation

```
import "std/c/cryptcrc32"
use cryptcrc32

create result = crc32(/* args */)
say result

create result = crc32List(/* args */)
say result

create result = verify(/* args */)
say result

create result = update(/* args */)
say result

create result = finalize(/* args */)
say result

create result = chunkCrc(/* args */)
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/cryptcrc32"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module cryptcrc32

function crc32(s text) returns number
end
```

```
function crc32List(data list number) returns list number
end

function verify(s text, expected number) returns bool
end

function update(crc number, byte number) returns number
end

function finalize(crc number) returns number
end

function chunkCrc(s text, chunkSize number) returns list number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/cryptcrc32` (tier complex, category complex). Exports 6 function(s): crc32, crc32List, verify, u

`import "std/c/cryptcrc32"` · `use cryptcrc32`

- `crc32(s text) ? number`
- `crc32List(data list number) ? list number`
- `verify(s text, expected number) ? bool`
- `update(crc number, byte number) ? number`
- `finalize(crc number) ? number`
- `chunkCrc(s text, chunkSize number) ? list number`

2. Installation & import

Add the module to your program:

```
import "std/c/cryptcrc32"
use cryptcrc32
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

? crc32(s text) ? number
? crc32List(data list number) ? list
? verify(s text, expected number) ? bool
? update(crc number, byte number) ? number
? finalize(crc number) ? number
? chunkCrc(s text, chunkSize number) ? list

4. Usage examples

```
import "std/c/cryptcrc32"
use cryptcrc32

create result = crc32(/* args */)
say result

create result = crc32List(/* args */)
say result

create result = verify(/* args */)
say result

create result = update(/* args */)
say result

create result = finalize(/* args */)
say result

create result = chunkCrc(/* args */)
say result
```

5. Best practices

? Prefer `import "std/c/cryptcrc32"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.

6. Appendix ? source

```
module cryptcrc32

function crc32(s text) returns number
end
```

```
function crc32List(data list number) returns list number
end

function verify(s text, expected number) returns bool
end

function update(crc number, byte number) returns number
end

function finalize(crc number) returns number
end

function chunkCrc(s text, chunkSize number) returns list number
end
end
```